Introduction du modèle du réseau neuronal

Introduction au perceptron

Ces travaux pratiques visent à consolider les concepts abordés lors de la séance de cours portant sur l'introduction du modèle neuronal. Ils s'étendent jusqu'à l'application du modèle à un robot différentiel. Cette série d'exercices se conclut par la rédaction d'**un compte rendu accompagné de scripts** permettant d'obtenir les résultats mentionnés. Ce rendu devra être déposé sur le site Moodle du cours.

1 Perceptron : implantation d'une porte logique

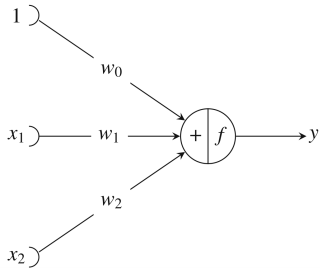

Fig. 1 : Perceptron à deux entrées

Le modèle du perceptron de la Fig. 1 est composé par deux entrées dénommées x_1 et x_2. Les valeurs de x_1 et x_2 peuvent être 0 ou 1. Assignez les poids de w_0,w_1 et w_2 de sorte à assurer une sortie y comme l'indiquent les tables de vérités ci-dessous :

Opérateur Logique OU (OR)		
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Opérateur Logique ET (AND)		
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Op. Logique OU-Excl. (XOR)		
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Tab. 1 : Tables de vérité des opérateurs logiques OU/ET et OU-Exclusif (XOR)

Afin de valider les poids proposés, implantez le modèle dans un script python en suivant la démarche proposée ci-dessous :

1. Définissez les variables x_1 et x_2 avec des valeurs 0 et/ou 1.
2. Dans une fonction retournant la valeur s, faites le calcul de l'expression :

$$s = w_0 + x_1 \cdot w_1 + x_2 \cdot w_2 \tag{1}$$

3. Dans une deuxième fonction prenant en entrée s et retournant y, vérifiez les conditions suivants :

$$\begin{array}{ll} \text{Si } s \geqslant 0\,, & y = 1 \\ \text{Si } s < 0\,, & y = 0 \end{array}$$

4. **Effectuez dans le script tous les cas indiqués dans le Tab. 1 sous un tracé et rapportez les poids w_0,w_1 et w_2 pour l'opérateur OU et ceux pour l'opérateur ET.**

Questions

- Est-ce qu'on peut obtenir des poids satisfaisant les valeurs des opérateurs logiques ET/OU ?
- Est-ce qu'on peut représenter à l'aide d'un perceptron l'opérateur OU-Exclusif ? Oui, Non. Pourquoi ?

2 Exercice d'application

Ton fin

Implantez les perceptrons dans votre robot Thymio II. Utilisez deux capteurs de proximité et redirigez leur mesure vers les entrées du perception x_1 et x_2. Utilisez la sortie du perceptron, y comme il est défini par le tableau ci-dessous :

- **Comportement A**

  Le robot se déplace vers l'avant si les deux capteurs de distance situés à l'arrière détectent la présence d'un obstacle.

- **Comportement B**

  Le robot se déplace vers l'avant seulement si les deux capteurs au sol détectent une surface.

3 Perceptron analogique

normalisé ÷ $0 < x \leq 1$

Les perceptrons peuvent fonctionner dans un domaine dit "analogique". Cela implique que les entrées et la sortie du perceptron pourront évoluer dans une gamme autre que celle du tout-ou-rien (TOR). **Afin de comprendre et de constater le fonctionnement de ce type de perceptron, implantez le modèle de la Fig. 2 dans le robot Thymio II.**

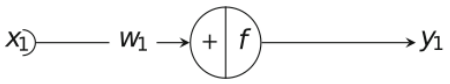

Fig. 2 : Modèle de perceptron à une entrée

- L'entrée du perceptron sera la valeur issue du capteur de proximité situé devant le robot.
- La sortie du perceptron sera dirigée vers :
  - la vitesse de moteurs afin de faire reculer le robot si un objet est détecté par le capteur.

Attention : La sortie du perceptron devra être proportionnelle,c-à-d, la vitesse de moteurs sera plus rapide si l'obstacle est plus proche.

- Implantez le modèle de perceptron à deux entrées (voir Fig. 3) en assignant des poids w_1 et w_2 différents. Illustrer que le capteur associé au poids plus fort a une plus ample influence sur la sortie.

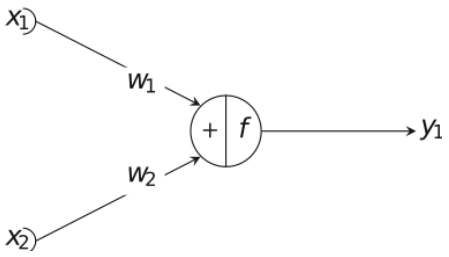

Fig. 3 : Modèle de perceptron à deux entrées

Interactions avec le Robot Thymio II

Il s'agit d'un système robotique simple, permettant la **téléopération** à partir d'un script python. En effet, le robot Thymio II sera utilisé dans le cadre de ce module sous un mode de contrôle à distance grâce à une liaison câblée ou sans fils (utilisation d'un dongle).

Le code d'exemple du List. 1 illustre la structure global à respecter lors du contrôle distant du robot. Ce code est disponible en téléchargement sur ce lien.

```
from thymiodirect import Thymio
from thymiodirect.thymio_serial_ports import ThymioSerialPort
import sys
import os
import time
def on_comm_error(error):
  #loss of connection: display error and exit
  print(error)
  os._exit(1) # forced exit despite coroutines

## Thymio callback
def obs(node_id):
  global done
  if not(done):
    if th[node_id]["button.center"]:
      print("button.center pressed")
      #Arret du robot
      th[node_id]["motor.left.target"] = 0
      th[node_id]["motor.right.target"] = 0
      done = True

################# ## Code principal #################
## Recherche du port
thymio_serial_ports = ThymioSerialPort.get_ports()
if len(thymio_serial_ports) > 0:
  serial_port = thymio_serial_ports[0].device
  print("Thymio serial ports:")
  for thymio_serial_port in thymio_serial_ports:
   print(" ", thymio_serial_port, thymio_serial_port.device)
try:
  th = Thymio(use_tcp=False,serial_port=serial_port,refreshing_coverage={"prox.horizontal", "prox.ground.reflected","prox.ground.ambiant",
  "prox.ground.delta","button.center"})
  #constructor options: on_connect, on_disconnect, on_comm_error,
  #refreshing_rate, refreshing_coverage, discover_rate, loop
except Exception as error:
  print(error)
  exit(1)

th.on_comm_error = on_comm_error
th.connect()
id = th.first_node()
done = False
th.set_variable_observer(id, obs)
print("Thymio executing code")
while not done:
  time.sleep(0.05) #Sampling routine
print("Thymio disconnecting")
th.disconnect()
```

Listing 1 : Code des premiers pas avec le Thymio II